

Sea Ice Classification

Deep Fusion Pipeline Workflow

End-to-end guide: raw data to trained model

Sentinel-2 optical + ICESat-2 ATL03 photon altimetry

This document traces every step of the deep fusion pipeline, from raw sensor data through preprocessing, patch extraction, model training, and final evaluation. It is a reproducibility guide and deliberately does not repeat the quantitative results and ablation analysis found in `project_summary.pdf`.

Five-Stage Pipeline Overview

Stage 1	Stage 2	Stage 3	Stage 4	Stage 5
Data Prep	Ground-Truth Gen	Train LSTM Branch	Train Fusion Model	Evaluate & Analyse

Key Scripts and Notebooks

<code>crop_all.py</code>	Extract 128x128 Sentinel-2 patches for every labeled point
<code>crop_csv.py</code>	Filter and align ICESat-2 CSV to matched patch coordinates
<code>crop_one_point.py</code>	Debug: extract and inspect a single patch
<code>segment_all.py</code>	Generate per-pixel ground-truth masks via HSV thresholding
<code>segment_one.py</code>	Debug: run segmentation on a single patch
<code>lstm_sweep.ipynb</code>	21-config hyperparameter sweep for the photon LSTM branch
<code>deep_fusion.ipynb</code>	Train the full U-Net + LSTM + SE fusion model

STAGE 1: DATA PREPARATION -- raw files -> aligned patches + CSV

Sentinel-2 input:	GeoTIFF scenes (*.tif) placed in S2_tiff/
ICESat-2 input:	ATL03 CSV files placed in IS2_Corrected_data/
Patch size:	128 x 128 pixels at 10 m resolution (1.28 km x 1.28 km)
Centering:	Each patch centered on a labeled ICESat-2 point (pix_x, pix_y)

```

raw Sentinel-2 GeoTIFF + ICESat-2 ATL03 CSV
    |                      |
    | crop_all.py          | crop_csv.py
    |                      |
    | outputs/             | IS2_Corrected_data/ (aligned)
    | (128x128 RGB patches) | (coord-matched CSV rows)

```

crop_one_point.py can be used to visually inspect a single patch before running the full batch.

Each patch filename encodes tile ID and photon-point index, enabling re-alignment to the CSV by (pix_x, pix_y). The CSV retains eight photon-height features used by the LSTM branch: h_cor_mean, h_cor_med, h_diff, rel_height_min_elev, height_sd, pcnth_mean, pcnt_mean, bcnt_mean/brate_mean.

STAGE 2: GROUND-TRUTH GENERATION -- patches -> per-pixel class masks

Script:	segment_all.py (single debug: segment_one.py)
Method:	HSV color thresholding with cloud and shadow removal
Output:	outputs_segmented/ -- one mask PNG per patch
Classes:	thick ice (red), thin ice (blue), open water (green)

```

outputs/ (128x128 RGB patches)
    |
    | segment_all.py
    |   | HSV threshold -> cloud/shadow mask -> 3-class pixel labels
    |   |
    |   | outputs_segmented/ (128x128 label masks)

```

Class encoding: R=255 -> thick ice | B=255 -> thin ice | G=255 -> open water. Pixels flagged as cloud or shadow are excluded from the training loss.

Training uses tiles T02CNA and T02CNC. The held-out evaluation tile T03CWT is geographically separated from both training tiles and is never used during any training or hyperparameter selection step.

STAGE 3: TRAIN LSTM PHOTON BRANCH -- lstm_sweep.ipynb -> runs/bilstm/

The photon branch is pretrained independently before fusion. A 21-configuration sweep selects the best hyperparameters.

Input sequence construction

For each labeled photon point at position i :

```
window = [segment i-2, i-1, i, i+1, i+2]    (seq_len = 5)
```

per-segment features (8 values each):

```
h_cor_mean    h_cor_med    h_diff    rel_height_min_elev
height_sd     pcnth_mean    pcnt_mean    bcnt_mean / brate_mean
```

```
input tensor shape: (batch, seq_len=5, features=8)
```

Sweep parameters (21 total configurations)

Focal alpha	[0.02,0.44,0.54] / [0.041,0.409,0.55] / [0.05,0.45,0.5] / [0.05,0.5,0.45]
Focal gamma	1.0 / 2.0 / 3.0 / 5.0
Hidden dim	32 / 64 / 96
Learning rate	5e-4 / 8.9e-4 / 1e-3 / 2e-3
Dropout	0.2 / 0.3 / 0.4 / 0.5
Sequence length	3 / 5 / 7
Random seed	7 / 42 / 123

Winner configuration

```
hidden = 96    dropout = 0.4    seq_len = 5    seed = 42
alpha = [0.05, 0.45, 0.60]    gamma = 2.0    lr = 8.9e-4
```

Outputs written to runs/bilstm/

```
runs/bilstm/
  confmat.png      -- confusion matrix on T03CWT
  test_metrics.json -- mIoU, per-class IoU, precision, recall, F1
  metrics.csv      -- per-epoch training and validation metrics
```

Focal loss with asymmetric alpha weights is critical: without it the model collapses and predicts thick ice for every pixel (dominant class). The saved checkpoint at the best validation mIoU epoch is loaded by Stage 4.

STAGE 4: TRAIN DEEP FUSION MODEL -- deep_fusion.ipynb -> runs/deep_fusion/

The fusion notebook loads the pretrained LSTM checkpoint from Stage 3, initializes a U-Net with ResNet-18 encoder, and trains the combined model end-to-end for 30 epochs on tiles T02CNA and T02CNC.

Model construction sequence inside deep_fusion.ipynb

1. Load pretrained LSTM checkpoint from runs/bilstm/
(hidden=96, dropout=0.4, alpha=[0.05,0.45,0.60], gamma=2.0)
2. Build U-Net with ResNet-18 encoder (ImageNet pretrained)
decoder output: feature map (16 channels, 128x128 spatial)
3. Build Squeeze-and-Excitation fusion head
 - project LSTM vector (96-d) -> 16 channels
 - broadcast across spatial grid (16, 128, 128)
 - concatenate with U-Net map -> (32, 128, 128)
 - SE block recalibrates channels (reduction ratio = 8)
 - 1x1 conv -> 3-class per-pixel logits
4. Optimizer learning rates
 - base lr for U-Net encoder + SE fusion head
 - 0.1 * base lr for LSTM branch (fine-tune, not freeze)
5. Train 30 epochs, save checkpoint at best validation mIoU

Why fine-tune the LSTM at 0.1x LR rather than freeze it?

Freezing the photon branch (fusion_v4) slightly under-performs fine-tuning it (deep_fusion). The reduced learning rate preserves photon representations learned in Stage 3 while allowing small adjustments for better alignment with the image branch features. This is the single most impactful design decision in the fusion architecture -- see project_summary.pdf ablation.

Outputs written to runs/deep_fusion/

```
runs/deep_fusion/
  confmat.png      -- confusion matrix on T03CWT
  test_metrics.json -- full per-class metrics
  loss_curve.png   -- training and validation loss per epoch
  summary_vs_all.csv -- comparison against all fusion variants
```

STAGE 5: EVALUATE AND ANALYSE -- held-out tile T03CWT -> metrics + plots

Evaluation is performed exclusively on tile T03CWT, which was withheld from all training and hyperparameter selection. The notebook computes all metrics from raw confusion-matrix pixel counts.

Metrics stored in test_metrics.json

miou	mean Intersection-over-Union across 3 classes
per_iou	per-class IoU [thick_ice, thin_ice, open_water]
pix_acc	overall pixel accuracy
precision	per-class $(TP / (TP + FP))$
recall	per-class $(TP / (TP + FN))$
f1	per-class harmonic mean of precision and recall
macro_f1	unweighted average F1 across 3 classes
cm	raw confusion matrix (absolute pixel counts, 3x3)

To reproduce results from scratch

```
# Stage 1-2: data preparation
python crop_all.py
python segment_all.py

# Stage 3: train LSTM photon branch
jupyter nbconvert --to notebook --execute lstm_sweep.ipynb

# Stage 4: train deep fusion model
jupyter nbconvert --to notebook --execute deep_fusion.ipynb

# Outputs are written automatically to runs/
# Recommended: set CUDA_VISIBLE_DEVICES=0 before launching
# Hardware: GPU >= 10 GB VRAM (tested on NVIDIA RTX A6000)
# Runtime: ~60-90 min per fusion training run (30 epochs)
```

Directory Reference

IS2_Corrected_data/	ICESat-2 ATL03 CSV inputs	(not tracked)
S2_tiff/	Sentinel-2 GeoTIFF scenes	(not tracked)
outputs/	Extracted 128x128 RGB patches	(not tracked)
outputs_segmented/	Per-pixel ground-truth masks	(not tracked)
runs/bilstm/	LSTM photon branch outputs	
runs/deep_fusion/	Deep fusion model outputs	
confusion_matrices/	Publication-ready confusion matrix PNGs	
archive/	Superseded fusion variants and LSTM runs	
notebook_output/	Intermediate outputs from exploratory work	
papers/	Reference literature	